```cpp
#include<bits/stdc++.h>
using namespace std;
class BitTrie
{
public:
    static constexpr int BITS = 31;       // for 32-bit ints use 31 down to 0
    vector<array<int,2>> nxt;
    vector<int> cnt;                      // count of numbers ending exactly at this node
    vector<int> pref;                     // count of numbers passing through this node
    int nodes = 0;

    BitTrie(int maxNodes = 1e6+1)
    {
        nxt.reserve(maxNodes);
        cnt.reserve(maxNodes);
        pref.reserve(maxNodes);
        newNode();
    }

    int newNode()
    {
        nxt.push_back({0,0});
        cnt.push_back(0);
        pref.push_back(0);
        return nodes++;
    }

    // insert x (f=+1) or remove x (f=-1)
    void insert(int x, int f = 1)
    {
        int u = 0;
        for (int b = BITS; b >= 0; --b)
        {
            int bit = (x >> b) & 1;
            if (!nxt[u][bit])
                nxt[u][bit] = newNode();
            u = nxt[u][bit];
            pref[u] += f;
        }
        cnt[u] += f;
    }

    void remove(int x) { insert(x, -1); }

    // exact count of x
    int count(int x) const
    {
        int u = 0;
        for (int b = BITS; b >= 0; --b)
        {
            int bit = (x >> b) & 1;
            int v = nxt[u][bit];
            if (!v || pref[v] <= 0) return 0;
            u = v;
        }
        return cnt[u];
    }

    // maximize x^y over all y in the trie
    int maxXor(int x) const
    {
        int u = 0;
        int ans = 0;
        for (int b = BITS; b >= 0; --b)
        {
            int bit = (x >> b) & 1;
            int want = bit ^ 1;
            if (nxt[u][want] && pref[nxt[u][want]] > 0)
            {
                ans |= (1 << b);
                u = nxt[u][want];
            } else
            {
                u = nxt[u][bit];
            }
        }
        return ans;
    }

    // count of elements â\211¤ x
    int leq(int x) const
    {
        int u = 0, res = 0;
        for (int b = BITS; b >= 0; --b)
        {
            int bit = (x >> b) & 1;
            if (bit == 1)
            {
                if(nxt[u][0])
                    res += pref[nxt[u][0]];
                u = nxt[u][1];
            }
            else
            {
                u = nxt[u][0];
            }
        }
        if(u)
            res += cnt[u];
        return res;
    }
};

int n;
vector<vector<int>> adj;
vector<bool> visited, is_cut;
vector<int> tin, low;
vector<pair<int,int>> bridges;
int timer = 0;
void dfs(int v, int p = -1) {
    visited[v] = true;
    tin[v] = low[v] = timer++;
    int children = 0;
    for (int to : adj[v]) {
        if (to == p) continue;
        if (visited[to]) {
            low[v] = min(low[v], tin[to]); // back edge
        } else {
            dfs(to, v);
            low[v] = min(low[v], low[to]);
            if (low[to] > tin[v]) {// â-\220 BRIDGE condition
                bridges.push_back({v, to});
            }
            if (p != -1 && low[to] >= tin[v]) {// â-\220 ARTICULATION POINT (non-root)
                is_cut[v] = true;
            }
            children++;
        }
    }
    if (p == -1 && children > 1) {// â-\220 ARTICULATION POINT (root)
```

```
        is_cut[v] = true;
    }
}

void find_all() {
    visited.assign(n, false);
    tin.assign(n, -1);
    low.assign(n, -1);
    is_cut.assign(n, false);
    bridges.clear();
    timer = 0;

    for (int i = 0; i < n; i++) {
        if (!visited[i]) dfs(i);
    }
}

class dsu_chan
{
public:
    int n;
    vector<int> par;
    vector<int> siz;
    dsu_chan(int n) : n(n), par(n), siz(n, 1)
    {
        iota(par.begin(), par.end(), 0);
    };
    int get(int x)
    {
        return (par[x] == x ? x : par[x] = get(par[x]));
    }
    void unite(int x, int y)
    {
        x = get(x), y = get(y);
        if(x == y)
            return;
        if(siz[x] > siz[y]) swap(x, y);
        par[x] = y, siz[y] += siz[x];
    }
    vector<vector<int>> group()
    {
        vector<vector<int>> g(n);
        for(int u = 0; u < n; u ++)
            g[get(u)].push_back(u);
        return g;
    }
};

class fenwick_tree_chan
{

public:
    vector<int> fenw;
    int n;
    int pw;

    fenwick_tree_chan() : n(0) {}
    fenwick_tree_chan(int n) : n(n)
    {
        fenw.resize(n);
        pw = (n == 0 ? 0 : 1ULL << (63 - __builtin_clzll(unsigned(n))));
    }

    // a[x] += v;
    void modify(int x, int v)
    {
        assert(0 <= x && x < n);
        while (x < n)
        {
            fenw[x] += v;
            x |= x + 1;
        }
    }

    /// sum of prefix [0, .. x]
    int query(int x)
    {
        ++ x;
        assert(0 <= x && x <= n);
        int v{};
        while (x > 0)
        {
            v += fenw[x - 1];
            x &= x - 1;
        }
        return v;
    }

    // Returns the length of the longest prefix (0 indexed) with sum <= c
    int max_prefix(int c)
    {
        int v{};
        int at = 0;
        for (int len = pw; len > 0; len >>= 1)
        {
            if (at + len <= n)
            {
                auto nv = v;
                nv += fenw[at + len - 1];
                if (!(c < nv))
                {
                    v = nv;
                    at += len;
                }
            }
        }
        assert(0 <= at && at <= n);
        return at;
    }
};

fenwick_tree_chan ds1(n), ds2(n);

void range_add(int l, int r, long long val) {
    ds1.modify(l, val);
    if (r + 1 < n) ds1.modify(r + 1, -val);

    ds2.modify(l, val * l);
    if (r + 1 < n) ds2.modify(r + 1, -val * (r + 1));
}
long long prefix_sum(int x) {
    if (x < 0) return 0;
    return (x + 1) * ds1.query(x) - ds2.query(x);
}
long long range_sum(int l, int r) {
    return prefix_sum(r) - prefix_sum(l - 1);
}
mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
#define unq(v) {sort(v.begin(), v.end()); v.erase(unique(v.begin(), v.end()), v.end());}

struct LCA {
    int n, LOG;
```

```
    vector<vector<int>> adj, up;
    vector<int> depth;
    void dfs(int v, int p) {
        up[v][0] = p;
        for (int j = 1; j < LOG; j++) {
            if (up[v][j-1] == -1)
                up[v][j] = -1;
            else
                up[v][j] = up[ up[v][j-1] ][j-1];
        }
        for (int to : adj[v]) {
            if (to == p) continue;
            depth[to] = depth[v] + 1;
            dfs(to, v);
        }
    }
    int kth_ancestor(int v, int k) {
        for (int j = 0; j < LOG; j++) {
            if (k & (1 << j)) {
                v = up[v][j];
                if (v == -1) return -1;
            }
        }
        return v;
    }
    int lca(int u, int v) {
        if (depth[u] < depth[v]) swap(u, v);
        int diff = depth[u] - depth[v];
        u = kth_ancestor(u, diff);
        if (u == v) return u;
        for (int j = LOG - 1; j >= 0; j--) {
            if (up[u][j] != up[v][j]) {
                u = up[u][j];
                v = up[v][j];
            }
        }
        return up[u][0];
    }
};
const int MAX=2e5+1;
bool v[MAX]={false};
vector<int>primes(1,2);
int len,sp[MAX];
void Sieve()
{
    for(int i=2;i<MAX;i+=2)sp[i]=2;
    sp[1]=1;
    for(int i=3;i<MAX;i+=2){
        if(!v[i]){
            primes.push_back(i);
            sp[i]=i;
            for(int j=i*i;j<MAX;j+=i){
                if(!v[j]){
                    v[j]=true;
                    sp[j]=i;
}}}}}
vector<pair<int,int>> factorize(int k)
{
    vector<pair<int,int>> pf;
    while (k > 1)
    {
        int p = sp[k],cnt = 0;
        while (k%p==0) k/=p,cnt++;
        pf.emplace_back(p, cnt);
    }
    return pf;
}
// phi[n] = n*(1-1/pi)
#include<bits/extc++.h>
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename K, typename V, typename Comp = std::less<K>>
using ordered_map = __gnu_pbds::tree<
    K, V, Comp,
    __gnu_pbds::rb_tree_tag,
    __gnu_pbds::tree_order_statistics_node_update
>;
template<class T> using
ordered_multiset = tree<T, null_type, less_equal<T>, rb_tree_tag, tree_o
rder_statistics_node_update>;
template <typename K, typename Comp = std::less<K>>
using ordered_set = ordered_map<K, __gnu_pbds::null_type, Comp>;
//  auto iterator = ordered_set().find_by_order(idx); // (0-indexed)
//  int num_strictly_smaller = ordered_set().order_of_key(key);

template <class S, auto op, auto e, class F, auto mapping>
struct simple_segtree
{
    int _n;
    int size;
    std::vector<S> d;
    simple_segtree() : simple_segtree(0) {}
    explicit simple_segtree(int n) { init(n); }
    explicit simple_segtree(const std::vector<S> &v)
    {
        _n = (int)v.size();
        size = 1;
        while (size < _n) size <<= 1;
        d.assign(2 * size, e());
        for (int i = 0; i < _n; ++i) d[size + i] = v[i];
        for (int i = size - 1; i >= 1; --i) d[i] = op(d[2*i], d[2*i+1]);
    }
    void init(int n)
    {
        _n = n;
        size = 1;
        while (size < _n) size <<= 1;
        d.assign(2 * size, e());
    }
    void set(int p, S x)
    {
        assert(0 <= p && p < _n);
        int pos = size + p;
        d[pos] = x;
        pos >>= 1;
        while (pos >= 1)
        {
            d[pos] = op(d[2*pos], d[2*pos+1]);
            pos >>= 1;
        }
    }
    S get(int p)
    {
        assert(0 <= p && p < _n);
        return d[size + p];
    }
    S prod(int l, int r)
    {
        assert(0 <= l && l <= r && r <= _n);
        if (l == r) return e();
        l += size; r += size;
        S sml = e(), smr = e();
        while (l < r)
```

```
        {
            if (l & 1) sml = op(sml, d[l++]);
            if (r & 1) smr = op(d[--r], smr);
            l >>= 1; r >>= 1;
        }
        return op(sml, smr);
    }
};
struct Hash {
    using ll = long long;
    const ll mod1 = 1000000007, mod2 = 1000000009;
    const ll base1 = 91138233, base2 = 97266353;
    int n;
    vector<ll> h1, h2, p1, p2;
    Hash(string s) {
        n = s.size();
        h1.resize(n+1, 0);
        h2.resize(n+1, 0);
        p1.resize(n+1, 1);
        p2.resize(n+1, 1);
        for (int i = 0; i < n; i++) {
            h1[i+1] = (h1[i]*base1 + s[i]) % mod1;
            h2[i+1] = (h2[i]*base2 + s[i]) % mod2;
            p1[i+1] = (p1[i]*base1) % mod1;
            p2[i+1] = (p2[i]*base2) % mod2;
        }
    }
    pair<ll,ll> get(int l, int r) {
        ll x1 = (h1[r+1] - h1[l]*p1[r-l+1] % mod1 + mod1) % mod1;
        ll x2 = (h2[r+1] - h2[l]*p2[r-l+1] % mod2 + mod2) % mod2;
        return {x1, x2};
    }
    // H.get(l, r) == RH.get(n - 1 - r, n - 1 - l); for pallindrome
};
class Trie
{
public:
    static constexpr int ALPHA = 26;
    vector<array<int,ALPHA>> nxt;
    vector<int> cnt, pref;
    int nodes;
    Trie(int mx = 1e6+1)
    {
        nxt.reserve(mx);
        cnt.reserve(mx);
        pref.reserve(mx);
        newNode(); // create root
    }
    int newNode()
    {
        nxt.push_back({});
        cnt.push_back(0);
        pref.push_back(0);
        return nodes++;
    }
    // insert s with frequency f (use f=-1 to remove)
    void insert(const string &s, int f = 1)
    {
        int u = 0;
        for (char ch : s) {
            int c = ch - 'a';
            if (!nxt[u][c]) {
                nxt[u][c] = newNode();
            }
            u = nxt[u][c];
            pref[u] += f;
        }
        cnt[u] += f;
    }
    // exact string count
    int count(const string &s) const
    {
        int u = 0;
        for (char ch : s)
        {
            int c = ch - 'a';
            if (!nxt[u][c] || pref[nxt[u][c]] <= 0)
                return 0;
            u = nxt[u][c];
        }
        return cnt[u]; // exact string count
        return pref[u]; // count of strings with prefix s
    }
};
```